Language models

attachment:78637862-31f7-4970-95de-000210e4a3a0:05_Language_models.pdf

- N-gram model
- perplexity
- N-gram model

Priority Notes

High Priority

1. Core definition of a language model

- A language model assigns probabilities to sequences of words or tokens.
- Two equivalent views appeared throughout the lecture:
- probability of a full sequence
- probability of the next word given previous words
- The lecturer emphasized that next-word prediction is the practical formulation used in many models.
- Key timestamps: `00:00:00 - 00:02:07`, `00:09:35 - 00:10:23`

2. Why n-gram models are useful and where they fail

- N-gram language models estimate probabilities by counting occurrences in a corpus.
- They are helpful for understanding the foundations of language modeling and can still work for simpler tasks.
- Their main limitation is sparsity:

- natural language has a huge vocabulary
- long word sequences rarely repeat enough times
- count-based estimates become unreliable for longer contexts
- Key timestamps: `00:05:02 - 00:09:24`

3. Evaluation idea: probability, log probability, and perplexity

- Direct probabilities of long sequences become tiny and are hard to work with numerically.
- The lecture connected sequence probability with log probabilities and related this to training loss ideas like cross-entropy.
- Perplexity was introduced as a standard normalized evaluation metric for language models.
- Lower perplexity means a better model fit to the test data.
- Key timestamps: `00:18:11 - 00:19:55`, `00:20:03 - 00:23:19`

4. Neural language models replace counting with learned representations

- Instead of relying only on counts, neural networks learn probability distributions over the vocabulary.
- Training is self-supervised:
- input is a context from the text
- target is the next word already present in the same text
- This was framed as a key transition from simple statistical models to modern language modeling.
- Key timestamps: `00:40:13 - 00:42:16`

Medium Priority

5. Feed-forward neural language model structure

- Fixed-size context window.
- Words are represented by IDs and looked up via an embedding matrix.
- Embeddings are concatenated and passed through hidden layers.

- Output layer uses softmax to produce a probability distribution over the vocabulary.
- This architecture is limited by the fixed window size.
- Key timestamps: `00:42:23 - 00:49:06`

6. Recurrent neural networks solve the fixed-window issue

- RNNs process words one by one and maintain a hidden state summarizing prior context.
- This allows modeling variable-length input sequences.
- The hidden state is updated from the current input and the previous hidden state.
- Key timestamps: `00:49:20 - 00:53:33`

7. Attention addresses the bottleneck of encoder-decoder models

- The main goal of attention is to let the decoder access all encoder states dynamically, not compress everything into one fixed vector.
- Attention computes a weighted combination of encoder states.
- Weights depend on which source positions are most relevant to the current decoding step.
- Key timestamps: `01:07:08 - 01:12:25`

Lower Priority

8. How attention weights are computed

- The lecture described attention as measuring relevance between the current decoder state and encoder states.
- A simple option is dot-product similarity.
- Scores are normalized with softmax and used as weights in a weighted sum of encoder states.
- More advanced scoring functions can include extra learned matrices or a small neural network.
- Key timestamps: `01:13:35 - 01:18:29`

9. Roadmap to the next lecture

- The lecturer positioned this lecture as preparation for transformer-based and modern large language models.
- Next lecture should move from these foundations toward more modern architectures.
- Key timestamps: `00:03:36 - 00:04:01`, `01:19:02 - 01:19:17`

Lecture Flow

1. What a language model is.
2. Count-based language modeling with n-grams.
3. Problems of sparsity and why we need better models.
4. Evaluation with probabilities, logs, and perplexity.
5. Feed-forward neural language models with embeddings and softmax.
6. Recurrent neural networks for variable-length context.
7. Attention as a mechanism to dynamically focus on encoder states.

Key Concepts To Remember

- joint probability
- conditional probability
- next-word prediction
- n-gram
- corpus counting
- data sparsity
- log probability
- cross-entropy
- perplexity
- embedding matrix
- softmax
- self-supervised training

- recurrent network
- hidden state
- attention
- dot-product similarity

Transcript Quality Note

- The transcript is usable and the main lecture structure is clear.
- Some segments contain recognition errors or repeated filler-like phrases, especially in more technical explanations, so formulas should be cross-checked against slides or course materials before using them in formal notes.